LEVEL 1 PROJECT

Temperature-Controlled PWM Fan Driver

with Alarm and Sensor-Fault Protection

PIC16F877A | LM35 | 16x2 LCD | PWM | UART

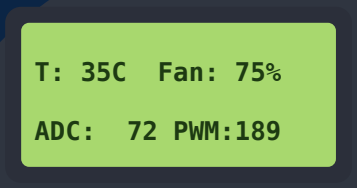




LM35



PWM FAN



16x2 LCD

Code port | Bit and register settings | Wiring | Maths | Testing

github.com/Amdspider/PIC16F877A-Temp-Controlled-PWM

Educational simulation project. Not a certified industrial safety device.

Welcome, future embedded engineer!

This handbook explains **every part** of the project **PIC16F877A Temperature-Controlled PWM** in simple steps. You do not need to be an expert. If you know a little C and have seen a microcontroller before, you can follow along. After reading you will know **what each line of the code does**, **which pin and register bit it uses**, and **how every number was calculated**.

1 Understand	2 Wire it	3 Read the code	4 Check the maths
The big picture: sensor in, fan speed out, alarm.	Which pin goes to which part, and why.	Every function, register and bit, in small parts.	Baud rate, PWM and ADC steps worked out by hand.

How to read this handbook

REMEMBER
An idea to keep in your head.

TIP
A shortcut or good habit.

WATCH OUT
A common mistake.

MATHS
How a number is calculated.

WHY?
Why the programmer chose it.

LEVEL UP
An idea to improve the project.

Before you start

- **You should know:** basic C (variables, if, while, functions) and what binary numbers are. Chapter 2 refreshes the rest.
- **Nice to have:** MPLAB X IDE with the XC8 compiler and Proteus, so you can run the project yourself (Chapter 7).
- **Tip:** read Chapter 1 and 2 first, then keep Chapter 5 open next to the source file `newmain.c`.

Contents

1 What This Project Does 3
2 Beginner Basics 5
3 Hardware and Wiring 7
4 Program Structure and Flow 9
5 Code Walkthrough 12
6 The Maths Behind the Numbers 26
7 Build, Simulate and Test 28
8 Level Up: Limits and Improvements 30
9 Practice and Quick Reference 31

1 What This Project Does

The big picture in plain English

Think about a machine in a factory. When it gets warm, a fan must cool it. When it gets *too* hot, a person must be warned. This project does that job automatically with one small chip, the **PIC16F877A**. The chip repeats five jobs again and again, many times every second:

#	Job	How it is done	Where in the code
1	**Measure** temperature	LM35 sensor gives 10 mV for every degree C. The ADC turns the voltage into a number.	`read_adc()`
2	**Show** it	A 16x2 LCD prints temperature, fan % and raw numbers.	`command()` `data()` `str_tx()`
3	**Cool** with a smart fan	PWM changes fan speed smoothly from 0 % (20 C) to 100 % (40 C).	`pwm_init()` `CCPR1L`
4	**Warn** when too hot	At 40 C or more: buzzer + red LED turn on, fan runs at full speed.	`BUZZER` `ALARM_LED`
5	**Report** and self-check	Sends numbers to a PC (UART). Detects a broken sensor after 3 bad readings.	`uart_str_tx()` `fault_count`

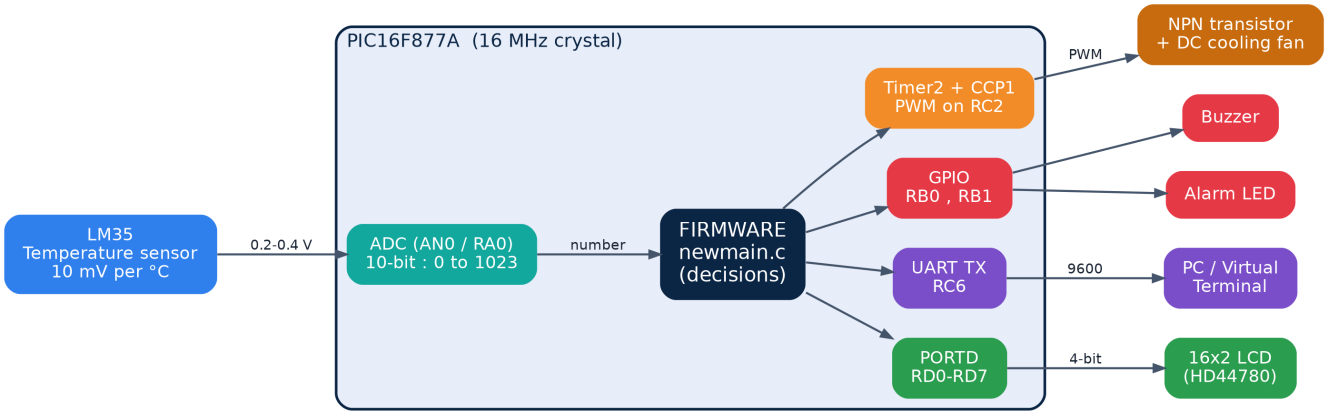


Figure 1.1 - Block diagram: signals flow from the sensor, through the chip, to the outputs.

The three working modes

The chip is always in one of three modes. The mode depends only on what the sensor reads. The LCD screens below are exactly what the program prints.

START-UP (2 seconds)

```
  SYSTEM START
Initializing...
```

NORMAL (below 40 C)

```
T: 35C  Fan: 75%
ADC:  72  PWM:189
```

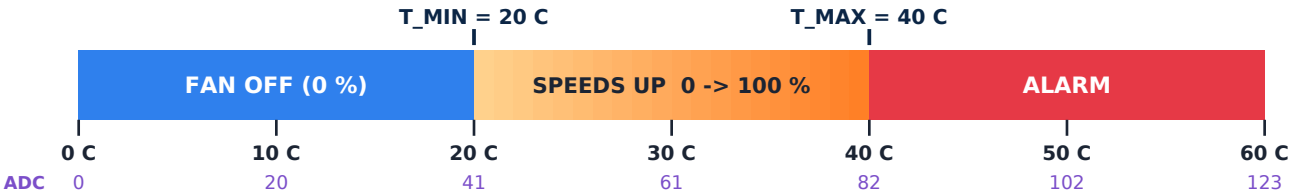
ALARM (40 C or more)

```
 ** ALARM! **
T: 42C  Fan:100%
```

SENSOR FAULT (3 bad readings)

```
SENSOR FAULT!
Check LM35 conn.
```

Figure 1.2 - The four LCD screens (every line is exactly 16 characters).



Row 3 shows the ADC count the chip reads at each temperature

Figure 1.3 - Temperature zones. Purple numbers are the ADC counts the chip reads at each temperature.

Key numbers to remember

20 C T_MIN: fan starts	**40 C** T_MAX: alarm + full fan	**16 MHz** crystal (HS oscillator)
9600 UART baud rate	**~4 kHz** PWM frequency (251 us)	**10-bit** ADC: 0 to 1023

2 Beginner Basics

Ten small ideas you need before reading the code

Each card is short on purpose. Read them once now. Come back later when a word in the code looks strange.

Microcontroller

A tiny computer on one chip. It has a brain (CPU), memory and **pins** to talk to the outside world. The PIC16F877A is 8-bit, has 40 pins, 5 ports (A to E), a 10-bit ADC, 3 timers, 2 CCP (PWM) modules and a UART.

Register and bit

A **register** is a memory box inside the chip, 8 bits wide. Each **bit** is a tiny switch: 0 = off, 1 = on. Writing a number to a register flips switches that control real hardware.

PORT and TRIS

`PORTx` holds the real level of the pins. `TRISx` sets the direction of each pin. **1 = Input, 0 = Output.** Trick: the digit 1 looks like the letter **I** (Input); the digit 0 looks like the letter **O** (Output).

ADC (analog to digital)

Turns a voltage (0 to 5 V) into a whole number (0 to 1023). 10 bits give 2^{10} = 1024 levels. One step is 5 V / 1023 = **4.89 mV**.

PWM (pulse width modulation)

The pin is switched ON and OFF very fast (about 4000 times a second). **Duty cycle** = the share of time it is ON. 25 % ON = slow fan, 100 % ON = full speed. It is like tapping a switch very quickly.

UART (serial)

Sends text one bit at a time on two wires: TX (send) and RX (receive). Both sides must use the same speed, called the **baud rate**. Here: 9600 bits per second.

LCD (HD44780)

A text display. RS = 0 means *command*, RS = 1 means *character*. The E pin (Enable) is a short pulse that tells the LCD "read the pins now". In **4-bit mode** each byte is sent as two halves called **nibbles**.

LM35 sensor

A 3-pin temperature sensor. Its output voltage is **10 mV per degree C**. 25 C gives 250 mV (0.25 V). 40 C gives 400 mV (0.40 V). No extra parts are needed.

Timer2

A hardware counter that counts clock ticks all by itself. Here it does two jobs: it sets the PWM period and it makes a simple time delay.

Binary and hex

Registers are written as hex (0x41) because it is short. One hex digit = 4 bits. 0x41 = 0100 0001. Bit 7 is on the left (MSB), bit 0 on the right (LSB).

Hex to binary table

Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Binary	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

MATHS

Example: `ADCON0 = 0x41;` → hex digits 4 and 1 → 0100 and 0001 → **0100 0001**. Now every bit can be matched with its name in the datasheet. This is how all register values in this handbook were built.

C operators used in this program

Operator	Name	Example	What happens
`&`	AND	`0x4E & 0xF0 = 0x40`	Keeps only the bits where the mask has 1. All other bits become 0. Used to **clear** bits.
`|`	OR	`0x40 | 0x01 = 0x41`	Sets a bit to 1 if it is 1 in either number. Used to **combine** two values.
`<<`	Shift left	`0x41 << 4 = 0x410`	Moves all bits to the left. Used to move the low nibble up to the high nibble position.
`= and ==`	Assign / compare	`x = 5 and x == 5`	**=** stores a value. **==** asks "are they equal?".
`||`	Logical OR	`a <= 0 || a >= 1023`	True if at least one of the two conditions is true.
`++`	Increment	`fault_count++`	Adds 1 to the variable.

3 Hardware and Wiring

Which part connects to which pin, and why

Parts list

Part	Job	Connected to
PIC16F877A	The brain. Runs the program.	+5 V, GND, crystal
LM35	Temperature sensor, 10 mV per degC.	OUT pin to RA0 (AN0)
16x2 LCD (HD44780)	Shows temperature, fan % and messages.	RS = RD0, E = RD1, RW = RD2, D4-D7 = RD4-RD7
NPN transistor + DC fan	Transistor works as a switch so the fan gets enough current.	Transistor base is driven from RC2 (CCP1)
Buzzer	Sound alarm.	RB0
Alarm LED + resistor	Light alarm.	RB1
16 MHz crystal + 2 capacitors	Gives the chip its clock.	OSC1 (pin 13) and OSC2 (pin 14)
Serial terminal	Shows UART text on the PC (Proteus Virtual Terminal or USB-TTL).	RC6 (TX) to terminal RX

Wiring diagram

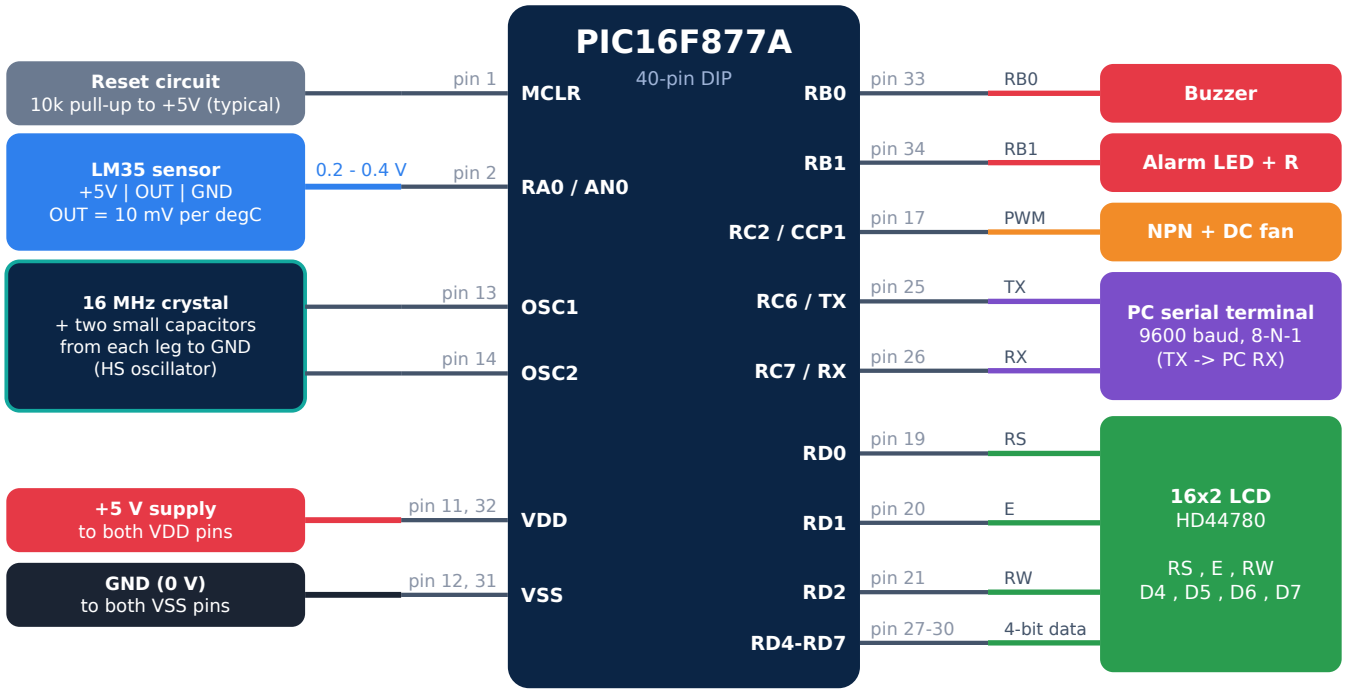


Figure 3.1 - All connections around the PIC16F877A. Resistor and capacitor values marked 'typical' follow common practice; check your own schematic.

Pin map: what, where and why

Pin	Name	Direction	Connected to	Why this pin?
2	RA0 / AN0	Input (analog)	LM35 OUT	RA0 is one of the pins that can be read by the ADC. Channel AN0.
17	RC2 / CCP1	Output	Fan transistor	PWM from the CCP1 module comes out on RC2 only. The pin is fixed by the chip.
25 / 26	RC6 TX / RC7 RX	Out / In	Serial terminal	The UART uses RC6 and RC7 only. Fixed by the chip.
33	RB0	Output	Buzzer	Free general pin. Any free pin would work.
34	RB1	Output	Alarm LED	Free general pin next to RB0.
19, 20, 21	RD0, RD1, RD2	Output	LCD RS, E, RW	Low nibble of PORTD carries the three LCD control lines.
27 - 30	RD4 - RD7	Output	LCD D4 - D7	High nibble of PORTD carries the data. This lets the code use a simple mask (0xF0).
13 / 14	OSC1 / OSC2	Clock	16 MHz crystal	Fixed oscillator pins.
11, 32 / 12, 31	VDD / VSS	Power	+5 V / GND	The chip has two of each. Connect both.

WHY?

Why not drive the fan straight from the pin? One PIC pin can give only about 25 mA. A fan needs much more. The pin therefore drives the **base** of an NPN transistor. The small base current switches a big current through the fan. Add a **flyback diode** across the fan so the voltage spike from the motor coil cannot hurt the transistor.

TIP

Look at the LCD wiring again. Data goes to RD4-RD7 and control goes to RD0-RD2. Because all five lines are on the **same port**, one line of code can change the data without touching the control lines.

4 Program Structure and Flow

How the file is organised and how the program runs

Files in the repository

File / folder	What it is
newmain.c	The whole C program (about 250 lines).
Proteus simulation (.DSN)	The circuit used to test the program without real hardware.
MPLAB X project folder	Project settings for the MPLAB X IDE and the XC8 compiler.
dist / ... / .hex	The compiled file that is loaded into the PIC (or into Proteus).
This handbook (PDF)	Explanations for students.

Order of the C file (top to bottom)

In C a function must be written **before** the line that uses it. That is why the file is built in layers: small tools first, the main program last.

1	**Configuration bits**	#pragma config lines. Tell the chip how to start.
2	**Includes and #defines**	Compiler help file, clock speed, pin names, thresholds.
3	**LCD functions**	lcd_nibble, command, data, str_tx, lcd_init
4	**UART functions**	uart_char_tx, uart_str_tx
5	**ADC function**	read_adc
6	**Timer2 delay**	delay
7	**Peripheral set-up**	adc_init, uart_init, pwm_init
8	**main()**	Port set-up, splash screen, then the endless loop.

All functions at a glance

Function	Job	Hardware used
`lcd_nibble(n)`	Puts 4 bits on the LCD data pins and pulses E.	PORTD
`command(cmd)`	Sends a command byte (RS = 0).	PORTD
`data(ch)`	Sends one character (RS = 1).	PORTD
`str_tx(str)`	Prints a whole text on the LCD.	PORTD
`lcd_init()`	Wakes up the LCD and sets 4-bit mode.	PORTD
`uart_char_tx(ch)`	Sends one character to the PC.	TXREG, TRMT
`uart_str_tx(str)`	Sends a whole text to the PC.	UART
`read_adc()`	Starts one conversion and returns 0 to 1023.	ADC, ADRESH, ADRESL
`delay()`	Waits about 25 ms using Timer2.	Timer2, TMR2IF

Function	Job	Hardware used
`adc_init()`	Sets ADC clock, channel and format.	ADCON0, ADCON1, TRISA
`uart_init()`	Sets 9600 baud, enables TX.	TXSTA, RCSTA, SPBRG
`pwm_init()`	Sets PWM period and starts at 0 %.	T2CON, PR2, CCP1CON, CCPR1L
`main()`	Runs everything.	All

Program flowchart



Figure 4.1 - Flowchart of newmain.c. Everything below 'START' runs once; the loop repeats forever.

State diagram: the three modes

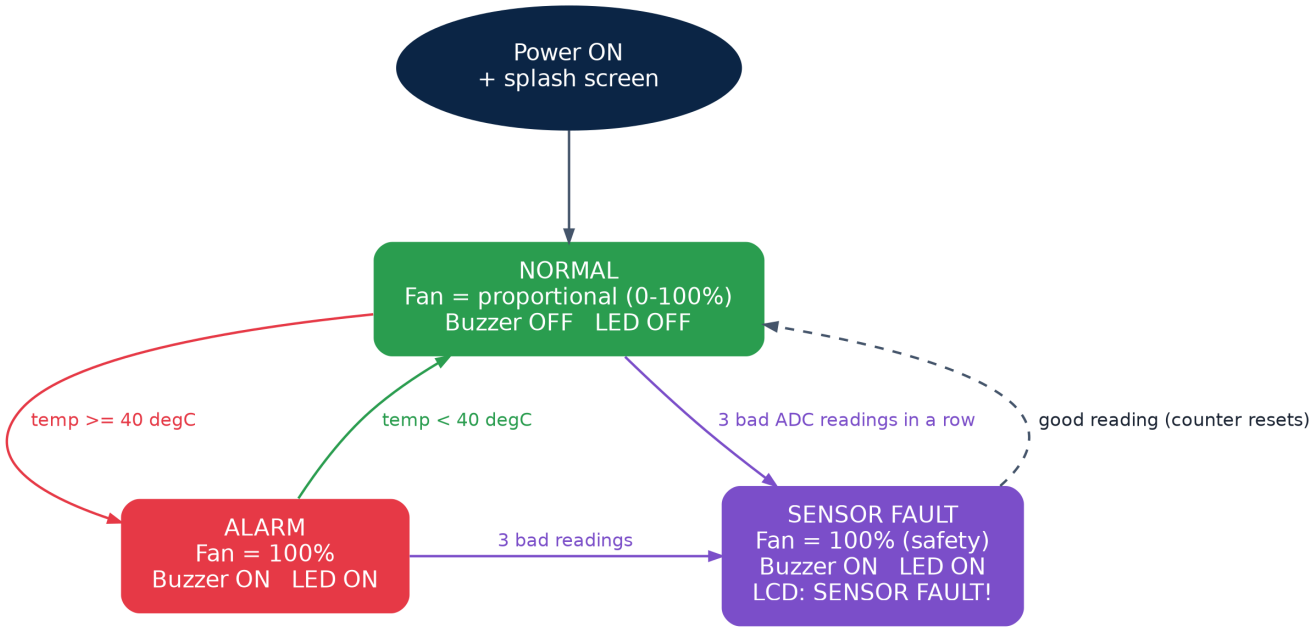


Figure 4.2 - The program moves between NORMAL, ALARM and SENSOR FAULT. There is no latch: when the problem goes away, the mode goes back.

REMEMBER

The program has **no memory of the alarm**. Each loop it looks at the sensor again and decides from scratch. If the temperature falls to 39 C the buzzer stops at the very next loop.

TIP

The fault check comes **before** the temperature check. A broken sensor can produce a false "0 C" or "500 C", so the program must decide first whether it can trust the reading.

5 Code Walkthrough

Every part of newmain.c: pins, registers, bits and the reason for each value

Code boxes below are copied from `newmain.c` (long comment banners are shortened to save space). After each box you will find a table that explains it **part by part**.

5.1 Configuration bits

These lines are not normal C code. The compiler writes them into special memory that the chip reads **at power-up**, before the program starts.

newmain.c - configuration bits

```
#pragma config FOSC = HS        // HS oscillator (16 MHz)
#pragma config WDTE = OFF       // Watchdog Timer disabled
#pragma config PWRTE = OFF      // Power-up Timer disabled
#pragma config BOREN = OFF      // Brown-out Reset disabled
#pragma config LVP = OFF        // Low-Voltage Programming disabled
#pragma config CPD = OFF        // Data EEPROM Code Protection off
#pragma config WRT = OFF        // Flash Write Enable off
#pragma config CP = OFF         // Flash Code Protection off
```

Setting	Meaning	Why this value?
FOSC = HS	High-speed crystal oscillator (for 4 to 20 MHz crystals).	The circuit has a 16 MHz crystal. A wrong choice means the chip runs at the wrong speed or does not start.
WDTE = OFF	Watchdog timer is off.	The watchdog resets the chip if the program does not clear it. This program never clears it, so it must be OFF.
PWRTE = OFF	No extra power-up delay (about 72 ms).	Not needed. The program waits 100 ms itself at the start of main().
BOREN = OFF	No reset when the supply voltage dips.	Keeps the simulation simple. A real product may switch this ON.
LVP = OFF	Low-voltage programming off.	Frees pin RB3 as a normal pin and avoids accidental programming mode.
CPD, WRT, CP = OFF	Code and EEPROM protection off.	Students can read and re-program the chip freely.

5.2 Includes and #define names

newmain.c - header and names

```
#include <xc.h>
#include <stdio.h>
#define _XTAL_FREQ 16000000UL   // 16 MHz HS oscillator

#define RS  RD0
#define RW  RD2
#define E   RD1

#define BUZZER    RB0
#define ALARM_LED RB1

#define T_MIN           20.0f   // Below this  -> fan OFF (PWM = 0)
#define T_MAX           40.0f   // At/above this -> ALARM, fan 100%
#define FAULT_THRESHOLD 3
```

Line	What it does	Why?
`#include <xc.h>`	Gives names such as PORTD, TRISC, ADCON0, RD0.	Without it the compiler does not know these register names.
`#include <stdio.h>`	Gives sprintf().	Used to turn numbers into text for the LCD and UART.
`_XTAL_FREQ 16000000UL`	Tells the compiler the clock speed.	__delay_ms() and __delay_us() calculate waiting loops from this number. UL = unsigned long.
`RS RD0 / E RD1 / RW RD2`	Short names for LCD control pins.	Code reads like a story: RS = 1; instead of RD0 = 1;
`BUZZER RB0 / ALARM_LED RB1`	Names for the alarm outputs.	To move the buzzer to another pin you change one line only.
`T_MIN, T_MAX`	Temperature limits. The f means a float number.	Change them here and the whole program follows.
`FAULT_THRESHOLD 3`	Bad readings in a row before a fault is declared.	One bad reading can be noise; three in a row is a real fault.

5.3 Port directions at the start of main()

newmain.c - main(): port set-up

```
void main(void)
{
    TRISC = 0x80;               // RC7=RX (input), RC6=TX (output)
    PORTC = 0x00;
    TRISB = 0x00;               // PORTB all output  (RB0=Buzzer, RB1=LED)
    PORTB = 0x00;               // All off
    TRISD = 0x00;               // PORTD all output  (LCD)
    PORTD = 0x00;

    __delay_ms(100);
```

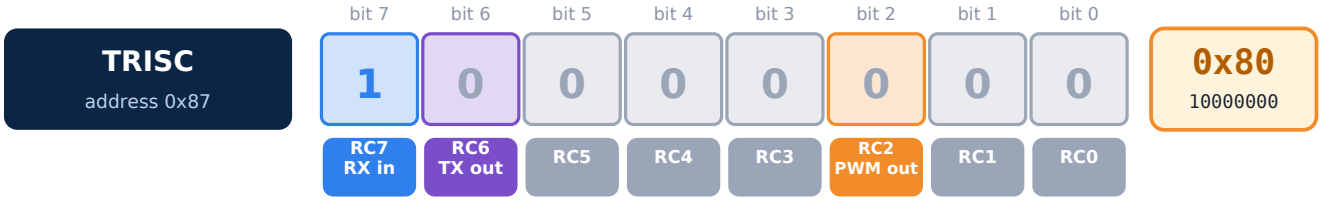


Figure 5.1 - TRISC = 0x80 = 1000 0000. Only bit 7 is 1 (input). Every other pin is an output.

Register	Value	Meaning	Why?
TRISC	0x80	RC7 input, all others output.	RC7 is the UART receive pin (input). RC6 is TX (output). RC2 must be an output for PWM.
TRISB	0x00	All PORTB pins output.	RB0 drives the buzzer and RB1 drives the LED.
TRISD	0x00	All PORTD pins output.	Every PORTD pin goes to the LCD (data and control).
PORTx = 0x00	0x00	All output pins start LOW.	Buzzer, LED and LCD lines start in a safe, quiet state.
__delay_ms(100)	-	Wait 100 ms.	Lets the power supply and the LCD settle before the first command.

REMEMBER
TRIS = direction (1 input, 0 output). **PORT** = value. Direction first, then value. TRISA is set to 0xFF inside adc_init() because RA0 must be an input for the sensor.

5.4 LCD driver (4-bit mode)

The LCD needs 8 data bits, but only 4 wires (D4 to D7) are connected. So every byte is sent **in two halves**: the high nibble first, then the low nibble. Each half is followed by a pulse on the E pin.

newmain.c - LCD functions

```
void lcd_nibble(unsigned char n)
{
    PORTD = (PORTD & 0x0F) | (n & 0xF0);   // D4-D7 on upper nibble of PORTD
    E = 1; __delay_us(50);
    E = 0; __delay_us(50);
}

void command(unsigned char cmd)
{
    RS = 0; RW = 0;
    lcd_nibble(cmd);            // High nibble first
    lcd_nibble(cmd << 4);       // Low nibble
    __delay_ms(5);
}

void data(unsigned char ch)
{
    RS = 1; RW = 0;
    lcd_nibble(ch);
    lcd_nibble(ch << 4);
    __delay_ms(2);
}

void str_tx(char str[])
{
    for (int i = 0; str[i] != '\0'; i++)
        data(str[i]);
}
```



Figure 5.2 - How the letter 'A' (0x41) travels to the LCD in two nibbles.

Part of the code	What it does	Why?
`PORTD & 0x0F`	Keeps the lower 4 bits of PORTD (RS, E, RW and the unused RD3). Clears the upper 4 bits.	The control lines must not change while data is placed.
`n & 0xF0`	Keeps only the upper 4 bits of n.	Only the upper nibble belongs on RD4-RD7.
`( ... ) | ( ... )`	Joins the two results.	Result: new data on RD4-RD7, control lines unchanged.
`E = 1 ... E = 0`	A pulse on Enable, 50 us high and 50 us low.	The LCD reads D4-D7 when E falls. 50 us is far longer than the minimum, so it is safe.
`RS = 0 (command) / 1 (data)`	Chooses what the byte means.	Command = "do something". Data = "draw this character".
`RW = 0`	Write mode.	The code never reads from the LCD, so RW is always LOW.
`cmd << 4`	Moves the low nibble up into the high nibble.	Example 0x28 << 4 = 0x280. The chip keeps only the low byte 0x80, so the upper nibble is 8.
`__delay_ms(5) / (2)`	Waits after a command / after a character.	The LCD needs time to finish. Clear needs about 1.5 ms. These delays are safely longer.
`str_tx()`	Loops through the text until it finds '\0' (end of text).	Every C string ends with a hidden zero. It tells the loop to stop.

Waking up the LCD: lcd_init()

newmain.c - lcd_init()

```
void lcd_init(void)
{
    __delay_ms(50);             // HD44780 power-on delay

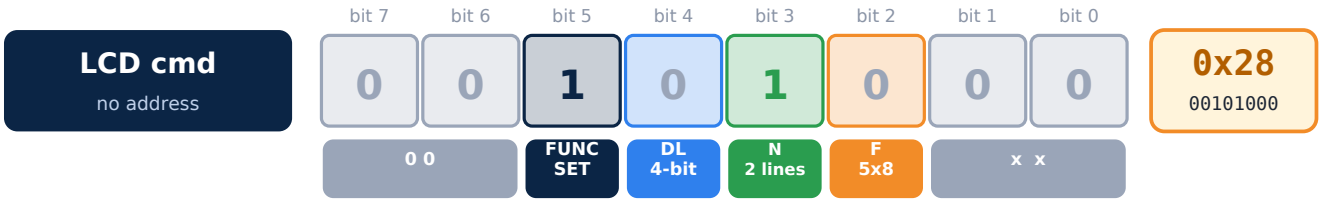
    RS = 0; RW = 0;
    lcd_nibble(0x30); __delay_ms(5);  // Wake pulse 1
    lcd_nibble(0x30); __delay_ms(1);  // Wake pulse 2
    lcd_nibble(0x30); __delay_ms(1);  // Wake pulse 3
    lcd_nibble(0x20); __delay_ms(2);  // Switch to 4-bit mode

    command(0x28);              // 4-bit, 2 lines, 5x8 font
    command(0x0C);              // Display ON, cursor OFF
    command(0x06);              // Entry: auto-increment, no shift
    command(0x01);              // Clear display
    __delay_ms(5);
}
```

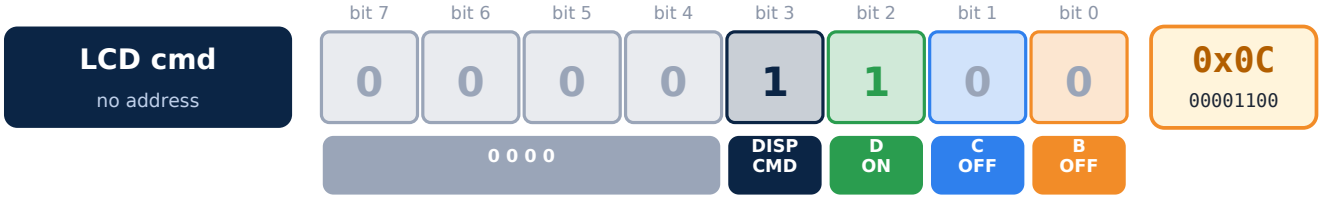
WHY?

Why send 0x30 three times? After power-up the LCD may be in 8-bit mode or in an unknown half-nibble state. Sending 0x30 three times forces it into a known state. Then 0x20 switches it to 4-bit mode. Only **one nibble** is sent in these four steps, because the lower four data lines are not wired.

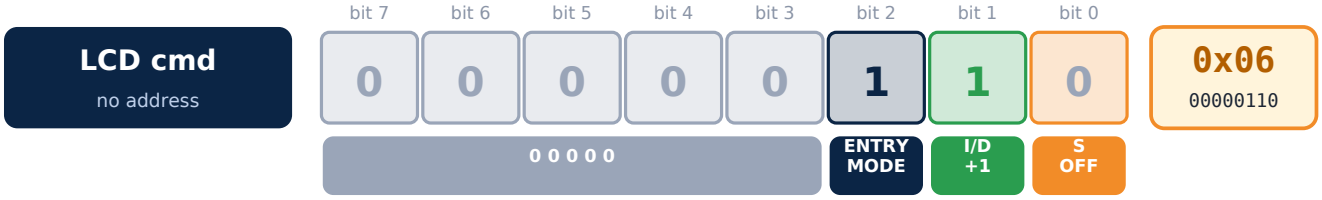
The three set-up commands, bit by bit. Each bit has a meaning on the LCD chip:



0x28 = Function set: DL = 0 (4-bit bus), N = 1 (two lines), F = 0 (5x8 dots).



0x0C = Display control: D = 1 (display on), C = 0 (no cursor), B = 0 (no blinking).



0x06 = Entry mode: I/D = 1 (cursor moves right after each character), S = 0 (screen does not shift).

Command	Meaning	Where used
0x01	Clear display. Needs more than 1.5 ms.	End of lcd_init() and after the splash screen.
0x80	Cursor to start of row 1 (address 0x00). Bit 7 = 1 means "set cursor address".	command(0x80) before writing row 1.
0xC0	Cursor to start of row 2. Row 2 starts at address 0x40, and 0x80 | 0x40 = 0xC0.	command(0xC0) before writing row 2.

5.5 ADC: reading the LM35

newmain.c - ADC functions

```
void adc_init(void)
{
    TRISA  = 0xFF;              // All PORTA pins as inputs (analog)
    PORTA  = 0x00;
    ADCON0 = 0x41;              // ADC ON, channel AN0, Fosc/8
    ADCON1 = 0x80;              // Right-justified result, all analog
}

int read_adc(void)
{
    GO = 1;
    while (GO == 1);            // Wait for conversion complete
    return (ADRESH << 8) | ADRESL;
}
```

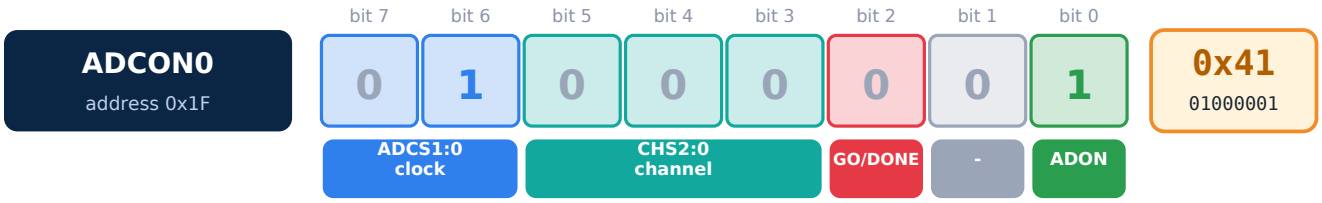


Figure 5.3 - ADCON0 = 0x41 = 0100 0001.

Bit(s)	Name	Value	What it means / why
7-6	ADCS1:ADCS0	01	ADC clock select. With ADCS2 = 0 (in ADCON1), 01 means Fosc/8.
5-3	CHS2:CHS0	000	Channel select. 000 = AN0 = pin RA0, where the LM35 is connected. (AN1 would be 001.)
2	GO/DONE	0	Start bit. The program sets it to 1 inside read_adc(). The chip clears it when finished.
1	(unused)	0	Not implemented.
0	ADON	1	1 = ADC module switched ON. If this is 0, the ADC does nothing.

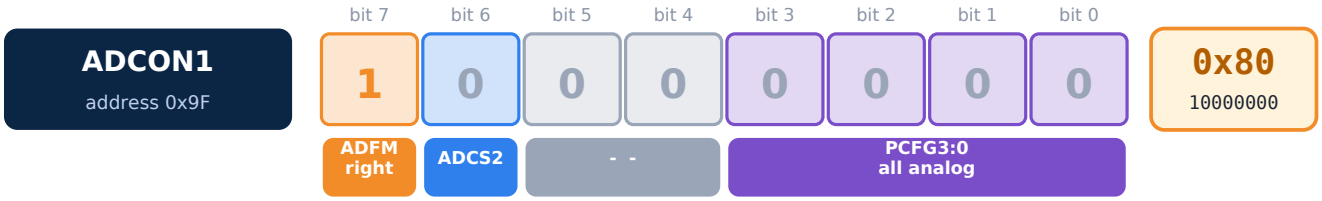


Figure 5.4 - ADCON1 = 0x80 = 1000 0000.

Bit(s)	Name	Value	What it means / why
7	ADFM	1	Result format. 1 = **right justified**: the 10-bit result sits in the lowest 10 bits of ADRESH:ADRESL.
6	ADCS2	0	Third clock bit. Together with ADCS1:0 = 01 it selects Fosc/8.
5-4	(unused)	00	Not implemented.
3-0	PCFG3:PCFG0	0000	Pin configuration. 0000 = all PORTA/PORTE pins analog; reference voltage = VDD (5 V) and VSS (0 V).

How read_adc() works

Step	Code	Explanation
1	`GO = 1;`	Sets the GO/DONE bit. The ADC starts measuring the voltage on AN0.
2	`while (GO == 1);`	Waits. When the conversion is finished, the chip sets GO back to 0 by itself and the loop ends.
3	`ADRESH << 8`	ADRESH holds the top 2 bits of the result. Shift it 8 places left to make room.
4	`| ADRESL`	ADRESL holds the lower 8 bits. OR joins the two bytes into one 10-bit number.

MATHS

Example: the chip measures a result of 584. In hex that is 0x0248. ADRESH = 0x02 and ADRESL = 0x48. The code computes (`0x02 << 8`) | 0x48 = 0x200 | 0x48 = 0x248 = 584. For a room temperature of 35 C the result is much smaller: **72** (ADRESH = 0x00, ADRESL = 0x48).

REMEMBER

ADC value is a ratio. result = (Vin / 5 V) x 1023. For the LM35: Vin = 0.01 V x temperature, so result = temperature x 2.046. At 40 C the ADC reads only about 82. The top of the scale (1023) is never reached by a healthy LM35 in this circuit.

5.6 UART: talking to the PC

newmain.c - UART functions

```
void uart_init(void)
{
    TXSTA = 0x24;               // TXEN=1, BRGH=1, async
    RCSTA = 0x90;               // SPEN=1, CREN=1
    SPBRG = 103;                // 9600 baud @ 16 MHz, BRGH=1
}

void uart_char_tx(char ch)
{
    while (TRMT == 0);          // Wait until transmit shift register empty
    TXREG = ch;
}

void uart_str_tx(char str[])
{
    for (int i = 0; str[i] != '\0'; i++)
        uart_char_tx(str[i]);
}
```

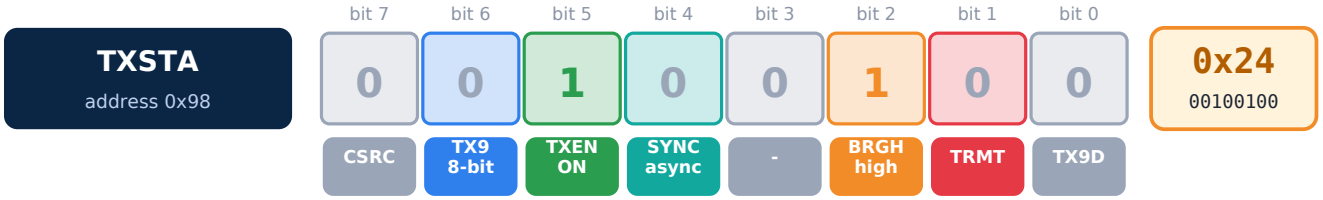


Figure 5.5 - TXSTA = 0x24 = 0010 0100.

Bit(s)	Name	Value	What it means / why
7	CSRC	0	Clock source. Only matters in synchronous mode. Ignored here.
6	TX9	0	9-bit transmit off. Normal 8-bit data.
5	TXEN	1	**Transmit enable.** 1 = the transmitter is switched ON.
4	SYNC	0	0 = **asynchronous** mode (no clock wire). This is standard UART.
2	BRGH	1	High baud-rate mode. The baud formula uses 16 instead of 64, which gives a more accurate result at 16 MHz.
1	TRMT	0	Read-only status flag: 1 means the transmit shift register is empty. The code reads it in uart_char_tx().

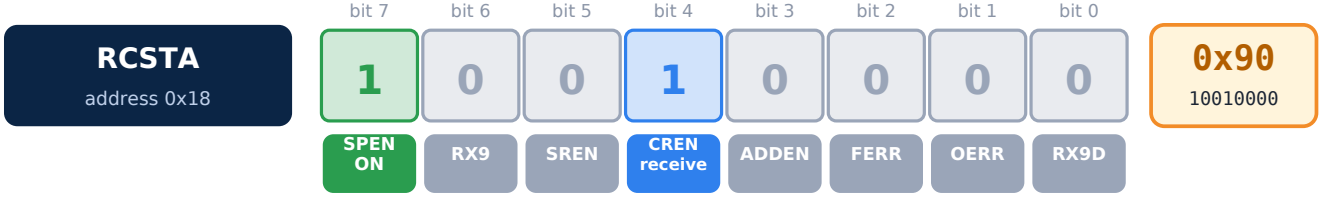


Figure 5.6 - RCSTA = 0x90 = 1001 0000.

Bit(s)	Name	Value	What it means / why
7	SPEN	1	**Serial port enable.** Connects the UART to pins RC6 (TX) and RC7 (RX). Without it the pins stay normal I/O.
4	CREN	1	Continuous receive enable. The receiver is on, although this program does not read anything from the PC.
others	-	0	9-bit receive off, no single receive, no address detect; the remaining bits are status flags.

MATHS

Where does 103 come from? With BRGH = 1 the datasheet formula is:
Baud = Fosc / (16 x (SPBRG + 1)) → SPBRG = Fosc / (16 x Baud) - 1
SPBRG = 16 000 000 / (16 x 9600) - 1 = 104.17 - 1 = 103.17 → **103** (nearest whole number).
Real baud = 16 000 000 / (16 x 104) = **9615 baud**. Error = +0.16 %. The PC accepts this easily.

What goes down the wire? One character becomes 10 bits: a START bit (0), eight data bits (least significant bit first) and a STOP bit (1). The line rests at 1 when idle. Each bit lasts 1 / 9600 = **104 us**.

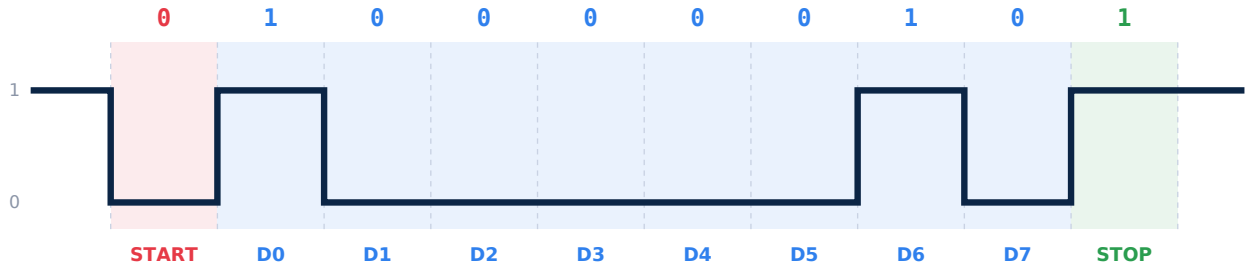


Figure 5.7 - The letter 'A' (0x41 = 0100 0001) on the TX wire. Data is sent bit 0 first, so the bits appear as 1 0 0 0 0 0 1 0.

WHY?

Why does uart_char_tx() wait for TRMT before writing? TXREG holds only one character. If the program wrote the next character too early, the previous one would be lost. Waiting for TRMT = 1 means the shift register is free. The cost: sending a 21-character line takes about 22 ms (21 x 1.04 ms), and the chip waits all that time.

5.7 PWM and the Timer2 delay

newmain.c - PWM set-up and delay()

```
void pwm_init(void)
{
    T2CON  = 0x05;          // Timer2 ON, prescaler = 4
    PR2    = 250;           // PWM period register
    CCP1CON = 0x0C;         // CCP1 in PWM mode
    CCPR1L  = 0;            // Initial duty = 0%
}

void delay(void)
{
    for (int i = 0; i < 100; i++)
    {
        while (TMR2IF == 0);
        TMR2  = 0;
        TMR2IF = 0;
    }
}
```

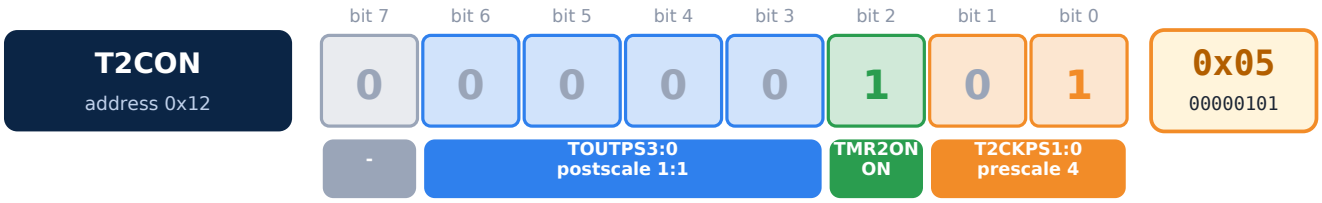


Figure 5.8 - T2CON = 0x05 = 0000 0101.

Bit(s)	Name	Value	What it means / why
6-3	TOUTPS	0000	Postscaler 1:1. The Timer2 flag is set on every match with PR2.
2	TMR2ON	1	**Timer2 ON.** The timer starts counting.
1-0	T2CKPS	01	Prescaler 1:4. Timer2 counts once for every 4 instruction cycles.

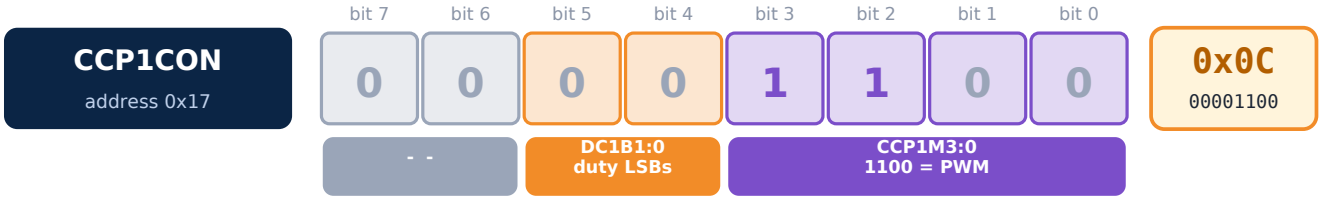


Figure 5.9 - CCP1CON = 0x0C = 0000 1100.

Bit(s)	Name	Value	What it means / why
5-4	DC1B1:DC1B0	00	The two lowest bits of the 10-bit duty value. They stay 00, so duty = CCPR1L x 4 (the program only uses the upper 8 bits).
3-0	CCP1M3:CCP1M0	1100	Mode select. 1100 = **PWM mode**. (0000 would switch the module off.)

Register	Value	Meaning
PR2	250	Timer2 counts from 0 up to 250, then restarts. That decides the PWM period (251 ticks).
CCPR1L	0 to 250	The duty register. Larger number = fan ON for longer. The program writes it every loop.

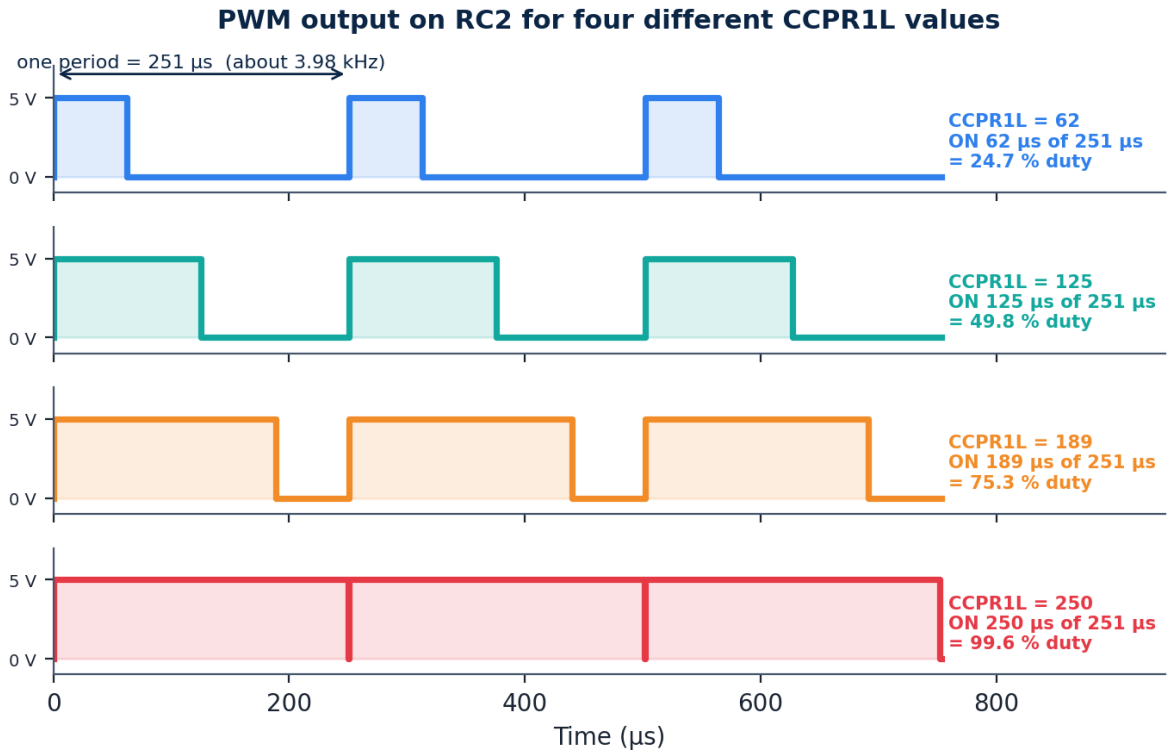


Figure 5.10 - Four PWM outputs. A bigger CCPR1L keeps the pin high for a longer part of every 251 us period.

MATHS

PWM period = (PR2 + 1) x 4 x Tosc x prescaler = 251 x 4 x (1 / 16 MHz) x 4 = 251 x 1 us = **251 us** → 3984 Hz (about 4 kHz).
ON time = (CCPR1L x 4) x Tosc x prescaler = CCPR1L x 4 x 0.0625 us x 4 = **CCPR1L x 1 us**.
So CCPR1L = 125 means ON for 125 us out of 251 us = 49.8 %.

WATCH OUT

At the top value CCPR1L = 250 the real duty is 250 / 251 = **99.6 %**, not exactly 100 %. The LCD shows 100 % because the program calculates fan % as pwm_val x 100 / 250. The difference is tiny and harmless.

How delay() makes about 25 ms

Step	Explanation
Timer2 tick	Fosc / 4 = 4 MHz instruction clock, divided by prescaler 4 = 1 MHz → one tick = **1 us**.
One Timer2 cycle	TMR2 counts 0 ... 250, which is 251 ticks = 251 us. At the match with PR2 the flag **TMR2IF** becomes 1.
Inside the loop	`while (TMR2IF == 0);` waits for the flag. `TMR2IF = 0;` clears it so the next cycle can be detected.
Repeat 100 times	100 x 251 us = 25 100 us = **about 25 ms**.

WHY?

Why use Timer2 and not __delay_ms()? Timer2 is already running for the PWM, so its flag is free to use as a time base. Because the delay watches the PWM timer, it also never disturbs the PWM output. Note: the whole loop takes about 130 ms because the LCD and UART writes are slow, so the display refreshes roughly 7 to 8 times per second.

5.8 The main program, step by step

Step A - start-up and splash screen

newmain.c - start-up

```
    lcd_init();
    uart_init();
    adc_init();
    pwm_init();

    command(0x80);  str_tx("  SYSTEM START  ");   // 16 chars
    command(0xC0);  str_tx("Initializing... ");   // 16 chars
    __delay_ms(2000);
    command(0x01);                                 // Clear display
    __delay_ms(5);

    char str[24];               // Scratch buffer
    int  fault_count = 0;       // Consecutive bad ADC readings
```

The four `..._init()` calls configure the hardware once. The splash text is shown for 2 seconds, then the screen is cleared. `str[24]` is a scratch area for `sprintf()`: the longest text is 16 characters (LCD) or 22 characters (UART line), plus 1 hidden end-of-text zero, so 24 bytes is safe. `fault_count` remembers how many bad readings happened in a row.

Step B - read the sensor and check it

newmain.c - loop part 1: read and fault check

```
    while (1)
    {
        int res = read_adc();   // 10-bit ADC result (0-1023)

        if (res <= 0 || res >= 1023)
            fault_count++;
        else
            fault_count = 0;    // Valid reading - clear counter

        if (fault_count >= FAULT_THRESHOLD)
        {
            CCPR1L    = 250;    // Force fan to full speed (safety)
            BUZZER    = 1;
            ALARM_LED = 1;

            command(0x80);  str_tx("SENSOR FAULT!   ");
            command(0xC0);  str_tx("Check LM35 conn.");

            uart_str_tx("\rSENSOR FAULT DETECTED\r");
            delay();
            continue;           // Skip rest of loop for this cycle
        }
```

Code	Explanation
`while (1)`	Endless loop. An embedded program never ends: when the power is on it keeps working.
`res <= 0 || res >= 1023`	A healthy LM35 at normal temperatures gives about 0.2 to 1 V, so res is roughly 40 to 205. A value of 0 (wire broken, shorted to GND) or 1023 (shorted to 5 V) is impossible, so it counts as a **bad reading**.
`fault_count++ / = 0`	A bad reading adds one. A good reading resets the counter. So 3 bad readings must come **in a row**.
`fault_count >= 3`	Fault confirmed. The chip protects the machine: fan to full speed, buzzer ON, LED ON, fault message on LCD and PC.

Code	Explanation
`continue`	Jumps back to the top of the loop at once. The temperature code below is skipped because the reading cannot be trusted.

WHY?

Why a safe fan at full speed during a fault? If the chip cannot measure, it does not know how hot the machine is. Cooling at full speed is the safest guess. This idea is called a **fail-safe** design.

Step C - convert to temperature and fan speed

newmain.c - loop part 2: maths

```
        float temp = (res * 5.0f / 1023.0f) * 100.0f;

        int pwm_val;
        if      (temp <= T_MIN) pwm_val = 0;
        else if (temp >= T_MAX) pwm_val = 250;
        else                    pwm_val = (int)((temp - T_MIN) * 250.0f / (T_MAX - T_MIN));
```

Code	Explanation
`res * 5.0f / 1023.0f`	Turns the ADC number back into volts (0 to 5 V). 5.0f means a float number.
`... * 100.0f`	The LM35 gives 10 mV = 0.01 V per degC, so degC = volts x 100.
`temp <= T_MIN`	20 degC or colder: fan stays off (0).
`temp >= T_MAX`	40 degC or hotter: fan at maximum (250).
`(temp - T_MIN) * 250 / (T_MAX - T_MIN)`	In between: a straight line from 0 at 20 degC to 250 at 40 degC. Example: 30 degC gives (10 x 250) / 20 = 125.
`(int)`	Cuts off the decimals. 189.88 becomes 189 (it does not round).

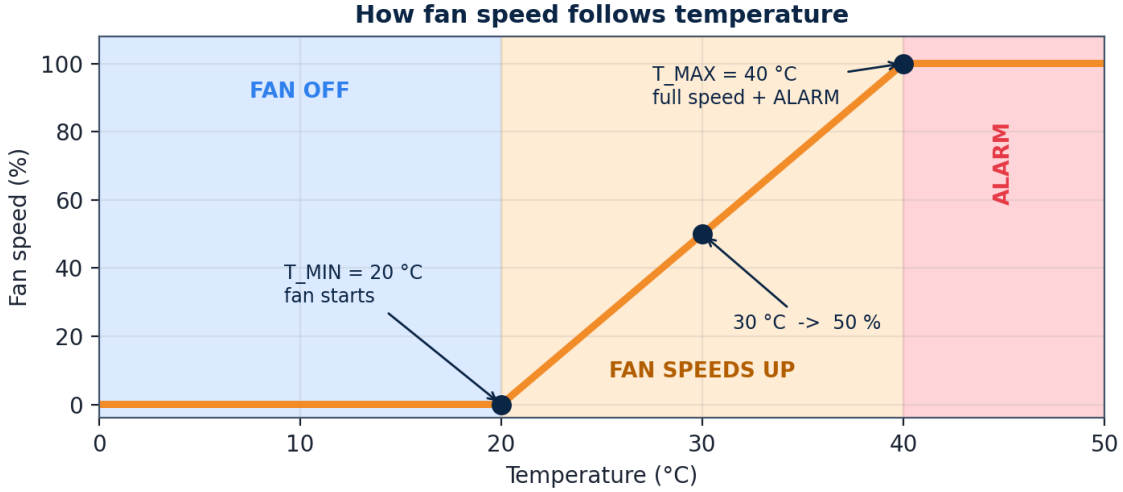


Figure 5.11 - The fan curve produced by the three lines above.

Step D - alarm or normal operation

newmain.c - loop part 3: outputs

```
        if (temp >= T_MAX)
        {
            BUZZER    = 1;
            ALARM_LED = 1;
            CCPR1L    = 250;

            command(0x80);
            str_tx(" ** ALARM! **   ");
            command(0xC0);
            sprintf(str, "T:%3dC  Fan:100%%", (int)temp);
            str_tx(str);
        }
        else
        {
            BUZZER    = 0;
            ALARM_LED = 0;
            CCPR1L    = pwm_val;

            int fan_pct = (pwm_val * 100) / 250;   // 0-100 %

            command(0x80);
            sprintf(str, "T:%3dC  Fan:%3d%%", (int)temp, fan_pct);
            str_tx(str);

            command(0xC0);
            sprintf(str, "ADC:%4d PWM:%3d", res, pwm_val);
            str_tx(str);
        }
```

Code	Explanation
`BUZZER = 1; ALARM_LED = 1;`	Writes a 1 to pins RB0 and RB1. Buzzer sounds, LED lights.
`CCPR1L = 250 / pwm_val`	This is **the** line that changes the fan speed. The PWM hardware reads CCPR1L and adjusts the ON time automatically.
`fan_pct = pwm_val * 100 / 250`	Converts the 0-250 duty to 0-100 %. Integer maths is used: the biggest value is 25 000, which fits in a 16-bit int (max 32 767).
`command(0x80) / (0xC0)`	Moves the LCD cursor to row 1 / row 2 before writing.
`sprintf(str, ...)`	Builds a text in the buffer, just like printf but into memory instead of a screen.
`(int)temp`	Whole degrees only. sprintf with float is large and slow on a small PIC, so the program avoids it.

The text formats: why every line is exactly 16 characters

Format	Pieces	Example output	Chars
`T:%3dC  Fan:%3d%%`	T: (2) + 3-wide number + C (1) + 2 spaces + Fan: (4) + 3-wide number + % (1)	`T: 35C  Fan: 75%`	16
`T:%3dC  Fan:100%%`	T: (2) + 3-wide number + C (1) + 2 spaces + Fan:100 (7) + % (1)	`T: 42C  Fan:100%`	16
`ADC:%4d PWM:%3d`	ADC: (4) + 4-wide number + space (1) + PWM: (4) + 3-wide number	`ADC:  72 PWM:189`	16

> **TIP**
> `%3d` means "print a whole number in a field 3 characters wide, right aligned". 5 becomes two spaces and 5. `%%` prints one real percent sign. Because every row always fills all 16 characters, new text covers old text completely, so the program never needs the slow clear-display command inside the loop. This also stops flicker.

Step E - telemetry and delay

newmain.c - loop part 4: UART and end of loop

```
        sprintf(str, "\radc=%d : pwm=%d \r", res, pwm_val);
        uart_str_tx(str);

        delay();
    }
}
```

After alarm or normal mode, one line is sent to the PC, for example `adc=72 : pwm=189`. The `\r` is a carriage return: it moves to the start of the line, so a terminal shows every reading on its own line. Then `delay()` waits about 25 ms and the loop starts again from `read_adc()`.

6 The Maths Behind the Numbers

How every value in the program was derived

Nothing in this program is a magic number. Each value comes from a short formula. Follow the boxes with a calculator once and you will own the method.

6.1 From temperature to ADC number

Step	Formula	Example at 35 C
1. Sensor voltage	Vout = 0.010 V x T	0.010 x 35 = 0.350 V
2. ADC number	res = Vout / 5 V x 1023	0.350 / 5 x 1023 = 71.6 → 72
3. Back to degC	T = res x 5 / 1023 x 100	72 x 5 / 1023 x 100 = 35.19 C
4. Whole degrees	(int)T	35

MATHS

Resolution: one ADC step = 5 V / 1023 = 4.887 mV = **0.489 C**. The chip cannot see changes smaller than about half a degree. That is why the calculated temperature moves in small steps (right chart below), and why the LCD may jump from 35 to 36 C.

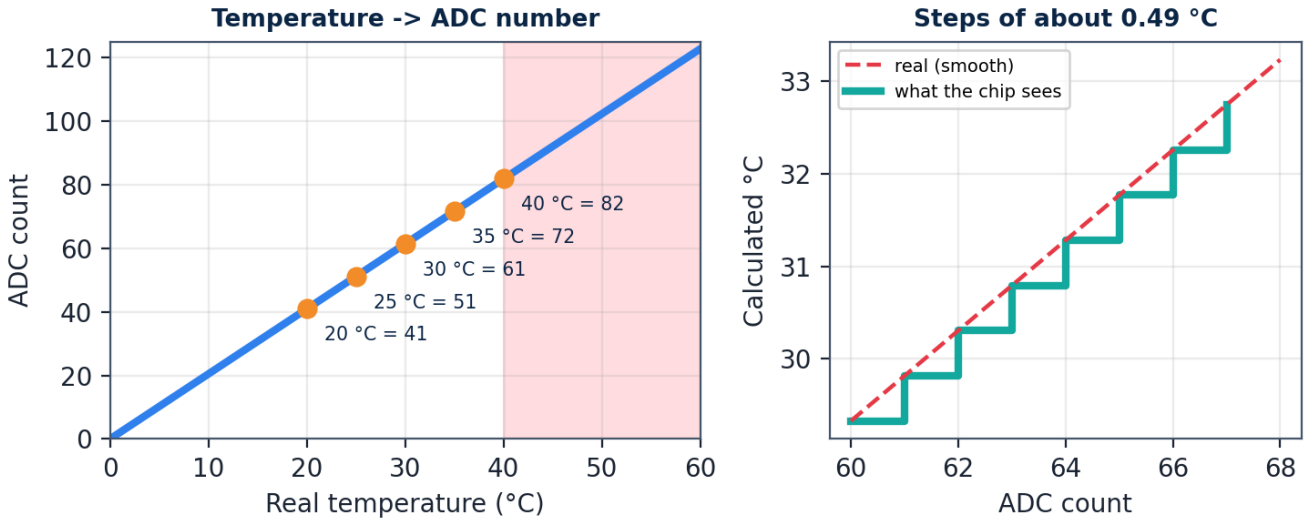


Figure 6.1 - Left: ADC count for a given temperature. Right: the chip only sees steps of 0.49 C.

6.2 From temperature to fan speed

MATHS

Straight-line formula (between 20 C and 40 C):
pwm_val = (T - 20) x 250 / (40 - 20) = (T - 20) x 12.5
fan % = pwm_val x 100 / 250 = (T - 20) x 5 → every degree above 20 C adds 5 % fan speed.
Example, T = 35.19 C: pwm_val = 15.19 x 12.5 = 189.9 → **189**; fan = 189 x 100 / 250 = 75.6 → **75 %**.

6.3 The complete table the program produces

ADC	Voltage	Temperature	pwm_val	Fan %	Mode
0	0.00 V	not trusted	250	100 %	SENSOR FAULT (after 3 in a row)
41	0.20 V	20.0 C	0	0 %	NORMAL
51	0.25 V	24.9 C	61	24 %	NORMAL
61	0.30 V	29.8 C	122	48 %	NORMAL
72	0.35 V	35.2 C	189	75 %	NORMAL
82	0.40 V	40.1 C	250	100 %	ALARM
100	0.49 V	48.9 C	250	100 %	ALARM
1023	5.00 V	not trusted	250	100 %	SENSOR FAULT (after 3 in a row)

TIP

To check any row yourself: pick an ADC number, multiply by 5, divide by 1023, multiply by 100. That is the temperature. Then apply the 20 C and 40 C rules from section 6.2.

6.4 Every timing and speed number in one place

Item	Formula	Result
Instruction clock	Fosc / 4 = 16 MHz / 4	4 MHz (one instruction = 0.25 us)
Timer2 tick	4 MHz / prescaler 4	1 MHz = **1 us** per tick
PWM period	(PR2 + 1) x 1 us = 251 x 1 us	**251 us** = 3984 Hz (about 4 kHz)
PWM ON time	CCPR1L x 1 us	CCPR1L = 189 → 189 us (75.3 %)
Maximum duty	250 / 251	99.6 %
delay()	100 x 251 us	**25.1 ms**
Baud rate setting	16 000 000 / (16 x 9600) - 1	SPBRG = 103 (real baud 9615, error 0.16 %)
One UART bit	1 / 9600	104 us
One UART character	10 bits x 104 us	1.04 ms
One LCD character	2 nibbles x 100 us + 2 ms	about 2.2 ms
One loop (approx.)	32 LCD chars (70 ms) + 2 commands (10 ms) + UART (22 ms) + delay (25 ms)	about 130 ms, 7 to 8 updates per second
ADC step	5 V / 1023	4.89 mV = 0.489 C

7 Build, Simulate and Test

From source code to a working circuit

Build the .hex file

Step	What to do
1	Open the project in **MPLAB X IDE**. The C compiler used is **XC8**.
2	Check that the device is PIC16F877A.
3	Press **Clean and Build**. When it says BUILD SUCCESSFUL, a .hex file appears in the dist folder.

Run in Proteus

Step	What to do
1	Open the .DSN simulation file.
2	Double-click the PIC chip. In **Program File** choose the .hex file.
3	Set **Processor Clock Frequency** to **16 MHz**. This must match _XTAL_FREQ in the code, otherwise delays and the baud rate are wrong.
4	Set the Virtual Terminal to **9600 baud, 8 data bits, no parity, 1 stop bit**.
5	Press Play. Change the LM35 temperature with the up/down arrows.

Test plan (expected results)

#	Set LM35 to	Expected ADC	Expected display and behaviour
1	15 C	31	NORMAL: LCD shows 15 C and fan 0 %, PWM 0, buzzer OFF
2	20 C	41	NORMAL: LCD shows 20 C and fan 0 %, PWM 0, buzzer OFF
3	30 C	61	NORMAL: LCD shows 29 C and fan 48 %, PWM 122, buzzer OFF
4	35 C	72	NORMAL: LCD shows 35 C and fan 75 %, PWM 189, buzzer OFF
5	39 C	80	NORMAL: LCD shows 39 C and fan 95 %, PWM 238, buzzer OFF
6	40 C	82	ALARM: buzzer + LED ON, fan 100 %, LCD shows 40 C
7	45 C	92	ALARM: buzzer + LED ON, fan 100 %, LCD shows 44 C
8	0 V (or sensor disconnected)	0	After 3 loops: SENSOR FAULT, fan 100 %, buzzer + LED ON
9	5 V on the ADC pin	1023	After 3 loops: SENSOR FAULT, fan 100 %, buzzer + LED ON
10	Fault, then back to 30 C	61	Normal screen returns. Buzzer and LED switch OFF.

What the serial terminal shows

Virtual Terminal (example)

```
adc=72 : pwm=189
adc=72 : pwm=189
adc=73 : pwm=194
SENSOR FAULT DETECTED
```

Troubleshooting

Problem	Likely cause	What to check
LCD is blank	No contrast, no power, wrong clock, hex not loaded.	LCD contrast (VEE) pin, +5 V, Processor Clock = 16 MHz, program file selected.
Strange letters on LCD	D4-D7 wires swapped or delays too short.	RD4 to D4 ... RD7 to D7 in order. Check RS, E and RW pins.
Temperature is wrong	ADC settings or sensor wiring.	ADCON1 = 0x80, LM35 OUT on RA0, 5 V reference, LM35 powered.
Fan never moves	PWM or transistor problem.	RC2 is output (TRISC bit 2 = 0), CCP1CON = 0x0C, transistor wiring, temperature above 20 C.
Buzzer on all the time	Temperature is at least 40 C or the sensor is in fault.	Read the LCD: ALARM or SENSOR FAULT tells you which.
No text on terminal	Wrong speed or wire.	9600 baud 8-N-1, RC6 (TX) to terminal RX, SPEN and TXEN set.

8 Level Up: Limits and Improvements

For students who want to go further

The program works well in simulation. A good engineer also knows its limits. These ideas turn a good student project into a more robust one.

Idea	Why it helps	How
Slower ADC clock	At 16 MHz, Fosc/8 gives an ADC clock period of 8 / 16 MHz = 0.5 us. The datasheet asks for at least 1.6 us, so a real chip may give less accurate readings. Proteus does not model this.	Use Fosc/32 (TAD = 2 us): change **ADCON0 = 0x81**. ADCS1:0 becomes 10 and ADCS2 stays 0.
Acquisition delay	The ADC input needs a short time to charge before a conversion.	Add **__delay_us(20);** before **GO = 1;** in read_adc().
True 100 % duty	CCPR1L = 250 gives 99.6 %. Full ON needs the duty to reach PR2 + 1.	Use **CCPR1L = 251** for the maximum value.
Alarm hysteresis	At exactly 40 C the buzzer could click ON/OFF with every small noise.	Switch ON at 40 C but switch OFF only below 38 C, using a flag variable.
Average the readings	Reduces noise on the LM35 signal.	Read the ADC 8 times, add them, divide by 8.
Integer maths	Float maths uses a lot of flash memory and time on an 8-bit PIC.	Use long integers: **temp_x10 = (long)res * 5000 / 1023** gives tenths of a degree.
Watchdog timer	Resets the chip if the program ever hangs. Good practice for industrial products.	Set WDTE = ON and call **CLRWDT()** once per loop.

LEVEL UP

Pick **one** idea, change the code, and test it in Proteus. Keep a copy of the working version first. Change one thing at a time so you know which change caused which result.

9 Practice and Quick Reference

Questions, challenges, cheat sheets and glossary

Check your understanding

#	Question	Answer
1	What ADC number does the chip read at 30 C?	30 C = 0.300 V. 0.300 / 5 x 1023 = 61.4, so **61**.
2	What are pwm_val and the fan % at exactly 30 C?	pwm_val = (30 - 20) x 250 / 20 = **125**. Fan % = 125 x 100 / 250 = **50 %**.
3	Why is TRISD = 0x00?	Every PORTD pin drives the LCD, so all eight must be outputs (0 = output).
4	Which ADCON0 value would read channel AN1 instead of AN0?	CHS2:0 = 001 sets bit 3. 0x41 | 0x08 = **0x49**.
5	What SPBRG value gives 19200 baud (BRGH = 1, 16 MHz)?	16 000 000 / (16 x 19200) - 1 = 51.08 → **51** (real baud 19231, error 0.16 %).
6	The temperature falls from 41 C to 39 C. What happens to the buzzer?	It switches **OFF** at the next loop. The program has no latch and decides again every time.
7	Why must every LCD text be exactly 16 characters?	So each new text completely covers the old one. The program then needs no slow clear command, and there is no flicker.
8	What does %% do inside sprintf?	It prints one percent sign (%). A single % would start a format code.
9	If PR2 were 124 (same prescaler), what would the PWM frequency be?	Period = (124 + 1) x 1 us = 125 us, so f = 1 / 125 us = **8 kHz**.
10	Why does the program check for a fault before it calculates the temperature?	A broken sensor gives a meaningless number. The program must decide first if the reading can be trusted.

Try it yourself

Challenge	Hint
Make the alarm start at 45 C and the fan reach 100 % at 45 C.	Change T_MAX only. Check that the maths still works.
Move the buzzer to RB2.	Change the #define and check that TRISB bit 2 is an output.
Send the data at 19200 baud.	New SPBRG value from Question 5. Change the terminal speed too.
Show the temperature in Fahrenheit on the PC.	F = C x 9 / 5 + 32. Add it to the UART sprintf().
Make the fan start at 25 C instead of 20 C.	Change T_MIN. Predict the new pwm_val at 30 C before you test.

Register cheat sheet

Register	Hex	Binary	Meaning
TRISA	0xFF	1111 1111	PORTA all inputs (RA0 = LM35)
TRISB	0x00	0000 0000	PORTB all outputs (RB0 buzzer, RB1 LED)
TRISC	0x80	1000 0000	RC7 input (RX); RC6 TX and RC2 PWM outputs
TRISD	0x00	0000 0000	PORTD all outputs (LCD)
ADCON0	0x41	0100 0001	Fosc/8 (with ADCS2 = 0), channel AN0, ADC ON
ADCON1	0x80	1000 0000	Right-justified result, all pins analog
TXSTA	0x24	0010 0100	Transmit ON, asynchronous, BRGH = 1
RCSTA	0x90	1001 0000	Serial port ON, receiver ON
SPBRG	103	0110 0111	9600 baud at 16 MHz, BRGH = 1
T2CON	0x05	0000 0101	Timer2 ON, prescaler 4, postscaler 1
PR2	250	1111 1010	PWM period = 251 ticks = 251 us
CCP1CON	0x0C	0000 1100	CCP1 in PWM mode
CCPR1L	0 - 250	variable	PWM duty (fan speed)

LCD command cheat sheet

Code	Name	Effect
0x30 (x3)	Wake-up	Resets the LCD to a known state at power-up.
0x20	4-bit select	Switches the LCD interface to 4 bits.
0x28	Function set	4-bit, 2 lines, 5x8 font.
0x0C	Display control	Display ON, cursor OFF, blink OFF.
0x06	Entry mode	Cursor moves right, no screen shift.
0x01	Clear	Erases the screen (needs more than 1.5 ms).
0x80	Set address	Cursor to row 1, column 1.
0xC0	Set address	Cursor to row 2, column 1.

Glossary

Word	Plain meaning
ADC	Analog-to-Digital Converter. Turns a voltage into a number.
Baud	Bits per second on a serial line.
CCP	Capture / Compare / PWM module. Here used as the PWM generator.
Duty cycle	The percentage of one PWM period for which the pin is HIGH.
Flag	A bit that tells the program that something has happened (for example TMR2IF).
Fosc	Oscillator (crystal) frequency: 16 MHz here.
Nibble	Half a byte: 4 bits.
Prescaler	A divider that slows a timer's clock.
PWM	Pulse Width Modulation. Fast ON/OFF switching to control average power.

Word	Plain meaning
Register	An 8-bit memory box that controls or reports hardware.
UART	Universal Asynchronous Receiver Transmitter. A simple serial port.
Fail-safe	A design that goes to the safest state when something fails.

ABOUT THIS PROJECT

Project: PIC16F877A-Temp-Controlled-PWM | **Source:** github.com/Amdspider/PIC16F877A-Temp-Controlled-PWM
Tools: MPLAB X IDE, XC8 compiler, Proteus. This is an educational simulation, not a certified industrial safety device.